A Scalable Multi-Node Architecture for Optimized LLM Inference Using Low-Cost Smartphone Clusters

MsC Juan Bernardo Calabrese[1], PhD Eduardo de la Vega Segura[2]

[1] Universidad Panamericana, Ciudad de México, México
jcalabrese@up.edu.mx

[2] Universidad Panamericana, Ciudad de México, México
evega@up.edu.mx

Abstract—This work investigates the feasibility of distributing a single large language model across multiple consumer smartphones to increase usable model capacity while maintaining fully local, network-isolated inference. The proof-of-concept platform uses two Google Pixel 9a smartphones running BiancaOS 0.6, a custom operating system based on Android and GrapheneOS. Inference is performed with llama.cpp using Qwen2.5 GGUF models quantized in Q4_K_M. Transformer layers are dynamically assigned according to available memory, inter-node communication uses USB 3.2, and a client computer accesses the cluster over Wi-Fi 6 through a local OpenAI-compatible HTTP API without participating in model computation. For consistency, all 1.5B, 3B, 7B, and 14B models were evaluated on the same two-device cluster. Prompt-processing and token-generation throughput were 30.53/12.70, 23.79/10.12, 18.39/9.76, and 15.00/8.15 tokens/s, respectively. The 14B configuration required approximately 12 GB of aggregate system RAM, leaving approximately 2 GB available, and therefore represents the principal memory-scaling result beyond the capacity of an individual 8 GB device. During operation, SoC and battery temperatures were approximately 45 °C and 30 °C, while combined wall-power consumption was approximately 25 W. The results demonstrate functional distributed execution of Qwen2.5-14B, although its 8.15 tokens/s generation rate remained below the predefined 10 tokens/s practical target.

1. INTRODUCTION

Large language models (LLMs) have become increasingly relevant for natural language processing, reasoning, coding, information retrieval, and interactive assistance. However, their growing memory and computational requirements continue to limit local deployment on resource-constrained devices. Smartphones provide an attractive platform for local inference due to their widespread availability, increasingly capable processors, integrated hardware accelerators, and high-speed connectivity. Nevertheless, the limited system memory of a single smartphone restricts the size of the models that can be executed efficiently [1].

This work investigates a distributed approach in which multiple consumer smartphones cooperate to execute a single LLM. Instead of requiring the complete model to fit within the usable memory of one device, transformer layers are distributed across multiple nodes, and intermediate activations are transferred between them during inference. Distributed model execution has previously been explored on edge-computing platforms [2]; this study examines its application to a homogeneous cluster of consumer smartphones operating locally and without direct Internet access.

The proposed experimental platform consists of two Google Pixel 9a smartphones equipped with 8 GB of RAM and Google Tensor G4 processors [3], running BiancaOS, a custom operating system based on Android and GrapheneOS. Inference is performed using llama.cpp [4] and quantized GGUF models from the Qwen2.5 family [5], ranging from 1.5B to 14B parameters. Model layers are dynamically assigned according to the available memory of each node. A conventional computer communicates with the cluster through a local Wi-Fi connection, while inter-node communication is performed through USB 3.2. All LLM computation remains within the smartphone cluster.

The objective of this study is to determine whether aggregating the memory resources of multiple smartphones can enable the practical execution of models that exceed the usable capacity of an individual device, while quantifying the performance costs introduced by distributed inference. The evaluation considers prompt-processing and token-generation throughput, aggregate memory utilization, thermal behavior, and wall-power consumption. Section 2 reviews related work and the main technical challenges. Section 3 describes the proposed architecture and experimental

methodology. Section 4 presents the experimental results, and Section 5 discusses the conclusions, limitations, and future extension of the architecture to larger smartphone clusters.

2. BACKGROUND, CHALLENGES AND PROBLEMS

2.1 Cost and Infrastructure Barriers

Running large language models locally requires substantial memory and computational resources. Although cloud platforms provide access to high-end accelerators, applications that require local processing must acquire and operate their own hardware. This creates a significant technological barrier for organizations that require private or offline inference.

The cost increases rapidly as larger amounts of accelerator memory are required. The NVIDIA GeForce RTX 5090 provides 32 GB of GDDR7 memory with a starting price of USD 1,999 and a total graphics power of 575 W [6]. Professional GPUs provide greater memory capacity: the RTX 6000 Ada offers 48 GB of ECC memory [7], while the RTX PRO 6000 Blackwell provides 96 GB of ECC memory [8] and is listed by NVIDIA at approximately USD 16,000 [9]. Data-center accelerators operate at an even higher scale. The NVIDIA H100 provides 80 GB of HBM3 memory [10], while the H200 provides 141 GB of HBM3e memory [11]. These devices are generally deployed inside specialized servers requiring high-capacity power delivery, cooling, networking, and rack infrastructure.

TABLE I. REPRESENTATIVE HIGH-END GPU RESOURCES

Accelerator	Memory	Maximum Power	Reference Price
GeForce RTX 5090	32 GB	575 W	USD 1,999 starting price
RTX 6000 Ada	48 GB	300 W	USD 10,000+
RTX PRO 6000 Blackwell	96 GB	600 W	USD 16,000+
H100 SXM	80 GB	up to 700 W	USD 30,000+ (standalone)
H200 SXM	141 GB	up to 700 W	USD 40,000+ (standalone)

These figures are not intended as a performance comparison with the smartphone architecture investigated in this work. They illustrate the economic and infrastructure requirements associated with obtaining large amounts of local accelerator memory. The accelerator itself is only one component of the total cost, as high-end systems may additionally require specialized servers, redundant power, cooling, and high-speed networking [12].

2.2 Local and Distributed LLM Inference

Memory capacity remains one of the principal constraints in local LLM inference. Quantization can substantially reduce model size, but increasingly large models may still exceed the memory available on a single consumer device. Previous research has demonstrated that smartphones can execute relatively large LLMs using specialized memory-management and scheduling techniques. PowerInfer-2, for example, demonstrated large-model inference on mobile hardware through coordinated use of heterogeneous smartphone resources [1].

Another approach is to distribute the model across multiple devices. Model-distributed inference partitions transformer layers among computing nodes and transfers intermediate activations during the forward pass. MDI-LLM demonstrated this approach on low-power edge devices, allowing models to be executed collectively when the memory capacity of an individual node is insufficient [2]. However, distributed execution introduces communication overhead, and previous work has shown that inter-device bandwidth and latency can become important performance constraints [13].

2.3 Challenges and Research Problem

Applying distributed LLM inference to smartphones introduces several practical challenges. The usable memory of each device is lower than its nominal RAM because the operating system, runtime, KV cache, activation buffers, and other processes compete for the same resources. Layer allocation must therefore consider dynamically available memory rather than physical memory alone.

Communication is another critical limitation. During autoregressive inference, intermediate data must be transferred repeatedly between devices, making latency and effective inter-node bandwidth important determinants of overall throughput. Smartphones also operate under tighter thermal and power constraints than workstation and data-center accelerators, which may affect sustained inference performance.

The objective of this work is not to reproduce the computational performance of high-end GPU infrastructure using smartphones. Instead, it investigates whether the aggregate memory and computing resources of multiple commodity smartphones can be organized into a practical, fully local LLM platform. The proposed architecture focuses on layer-based model partitioning, dynamic allocation according to available memory, high-bandwidth wired communication between nodes, and operation without direct Internet access.

3. METHODOLOGY

3.1 Experimental Platform

The experimental platform consists of two Google Pixel 9a smartphones, each equipped with 8 GB of RAM, 128 GB of internal storage, and a Google Tensor G4 system-on-chip [3]. Both devices run BiancaOS 0.6, an experimental custom operating system based on Android and GrapheneOS. The cluster operates without direct Internet access. A conventional computer acts only as a client and does not participate in LLM computation.

3.2 Models and Inference Runtime

Inference is performed with llama.cpp [4] using GGUF models from the Qwen2.5 family [5], quantized in Q4_K_M. Four model sizes are evaluated: 1.5B, 3B, 7B, and 14B parameters. For consistency, all four models are executed using the same two-Pixel distributed architecture. The 1.5B, 3B, and 7B configurations can fit within the memory capacity of an individual 8 GB device, while the 14B model is used as the principal capacity-scaling case. The reference configuration uses Vulkan acceleration on the Mali-G715 GPU, four CPU threads, a batch size of 512, a microbatch size of 64, Flash Attention, and a 4,096-token context window.

3.3 Distributed Architecture

The two smartphones operate as a distributed cluster without a permanently assigned master node. The LLM is partitioned at transformer-layer boundaries, and layers are assigned dynamically according to the usable memory available on each device. In the current implementation, both smartphones store the complete GGUF model, but each node loads only its assigned layers into RAM. Physical model sharding is reserved for future work. The client computer communicates with the cluster over Wi-Fi 6 through a local OpenAI-compatible HTTP API. Communication between the two Pixel devices is performed directly through USB 3.2. A 10 Gbit/s effective inter-node bandwidth budget is used for experimental design.

3.4 Experimental Procedure

All experiments use a 4,096-token context window and a maximum output of 256 tokens, with temperature = 0, top-p = 1, and a fixed seed. A fixed internal prompt set is used for all model sizes, and each controlled condition is repeated ten times under the same two-device configuration. Thermal behavior is monitored during testing, with reference values of approximately 45 °C for the SoC and 30 °C for the battery. Combined electrical consumption of the two Pixel devices is measured externally at the wall. For the 14B model, functional success requires stable loading and inference without out-of-memory or communication failures, while practical success additionally requires at least 10 tokens/s in generation throughput.

3.5 Measurements

The evaluation records prompt-processing speed, token-generation throughput, aggregate system-memory utilization, SoC and battery temperature, and combined wall-power consumption. Results from the ten repetitions are summarized using mean, median, standard deviation, minimum, and maximum values. For the 14B configuration, used and available system RAM are additionally recorded to quantify the memory-scaling behavior of the two-device cluster.

3.6 Network Isolation

During all experiments, WAN connectivity is disabled and the two smartphone nodes have no direct Internet access. The client computer communicates with the cluster only through the local Wi-Fi 6 network. The security scope of this

work is therefore limited to local, network-isolated inference and does not address compromised devices, malicious local clients, physical attacks, or broader threat models.

4. RESULTS

4.1 Distributed LLM Performance

All four Qwen2.5 Q4_K_M models were evaluated on the same two-Pixel 9a cluster to maintain a consistent execution architecture across model sizes. Prompt-processing and token-generation throughput decreased progressively as model size increased, as shown in Table II.

TABLE II. DISTRIBUTED LLM PERFORMANCE ON TWO PIXEL 9A DEVICES

Model	Prompt Processing	Token Generation	Quantization
Qwen2.5-1.5B	30.53 t/s	12.70 t/s	Q4_K_M
Qwen2.5-3B	23.79 t/s	10.12 t/s	Q4_K_M
Qwen2.5-7B	18.39 t/s	9.76 t/s	Q4_K_M
Qwen2.5-14B	15.00 t/s	8.15 t/s	Q4_K_M

Qwen2.5-1.5B achieved 30.53 tokens/s in prompt processing and 12.70 tokens/s in generation. The 3B model achieved 23.79 and 10.12 tokens/s, respectively; the 7B model achieved 18.39 and 9.76 tokens/s; and the 14B model achieved 15.00 tokens/s in prompt processing and 8.15 tokens/s in generation. The 14B configuration therefore achieved functional distributed execution but remained below the predefined practical generation target of 10 tokens/s.

The 1.5B, 3B, and 7B models were also distributed across both smartphones for experimental consistency, even though their memory requirements allow execution on an individual 8 GB Pixel 9a.

4.2 Operating Conditions

The experiments also characterized the thermal, electrical, and memory operating conditions of the two-device platform. The values reported in Table III correspond to the distributed test environment, including the 14B memory-scaling run.

TABLE III. EXPERIMENTAL OPERATING CONDITIONS

Parameter	Measured Value
14B system RAM used	≈12 GB
14B free RAM	≈2 GB
SoC temperature	≈45 °C
Battery temperature	≈30 °C
Combined wall power	≈25 W
Inter-node interface	USB 3.2
Client interface	Wi-Fi 6

During the 14B distributed run, approximately 12 GB of aggregate system RAM were in use across the cluster, with approximately 2 GB remaining available. Combined wall-power consumption of both Pixel 9a devices was approximately 25 W.

4.3 Capacity Scaling Result

The 14B configuration is the principal capacity-scaling result of the study. Unlike the 1.5B, 3B, and 7B models, its memory requirement exceeds the capacity available on an individual 8 GB Pixel 9a. Distributed execution across the two devices completed successfully while using approximately 12 GB of aggregate system RAM and leaving approximately 2 GB available.

The 14B model achieved 15.00 tokens/s in prompt processing and 8.15 tokens/s in generation. This demonstrates functional execution through resource aggregation, although generation performance remained below the predefined practical target of 10 tokens/s.

5. CONCLUSIONS

This work investigated the use of consumer smartphones as a distributed platform for fully local LLM inference, with the objective of increasing practical model capacity beyond the memory limitation of a single mobile device. The proposed architecture combines two Pixel 9a smartphones running BiancaOS 0.6, transformer-layer partitioning, memory-aware workload distribution, USB 3.2 inter-node communication, and a Wi-Fi 6 client interface.

Using the same two-device cluster for all tests, Qwen2.5-1.5B, 3B, 7B, and 14B Q4_K_M achieved generation rates of 12.70, 10.12, 9.76, and 8.15 tokens/s, respectively. Prompt-processing rates were 30.53, 23.79, 18.39, and 15.00 tokens/s. The measurements provide a consistent distributed baseline and show the expected reduction in throughput as model size increases.

The principal result is the successful distributed execution of Qwen2.5-14B. BiancaOS reported approximately 12 GB of aggregate system RAM in use across the cluster, with approximately 2 GB available during the run. The model therefore exceeded the memory capacity of an individual 8 GB node while remaining operational across the two-device cluster. The 14B model achieved functional success, although its 8.15 tokens/s generation rate remained below the predefined practical target of 10 tokens/s.

These results should be interpreted as an initial software baseline rather than a performance ceiling for smartphone-class hardware. BiancaOS 0.6 and the distributed runtime remain experimental, and substantial additional throughput may be available through improved layer placement, memory management, pipeline scheduling, buffer reuse, lower-overhead USB transfers, and more complete use of the available GPU and NPU resources. The present 14B result therefore identifies a working capacity point, not the maximum performance of the architecture.

Future work will address both performance and cost. The software will be optimized for additional nodes, heterogeneous platforms, physical model sharding, and improved accelerator utilization. At larger deployment volumes, the architecture also does not require complete retail smartphones: a production node could be reduced to a board-level implementation centered on smartphone-class compute hardware, memory and storage, power and thermal support, and the USB interconnect, while omitting handset components that do not contribute to inference, such as the display, cameras, enclosure, and other consumer-facing subsystems. Combined with volume procurement and simplified hardware, this approach could substantially reduce cost per node while preserving the local, network-isolated distributed architecture. These optimizations are future engineering directions and were not measured in the present study.

6. BIOGRAPHIES

Juan Bernardo Calabrese is a technology innovation leader and researcher specializing in artificial intelligence, computer vision, embedded systems, and hardware-software integration. His work includes nanosatellite development, satellite imagery analysis, GAN-based super-resolution, industrial computer vision, and applied AI systems. He has also worked on computer vision and satellite-image analysis for public-sector applications in Álvaro Obregón, including urban monitoring, infrastructure analysis, and decision-support use cases. He holds a B.Sc. in Mechatronics Engineering and an M.Sc. in Science from Universidad Panamericana and is currently pursuing a Ph.D. in Artificial Intelligence.

LinkedIn Bernardo Calabrese

Eduardo de la Vega Segura holds a Ph.D. in Engineering from Universidad Panamericana, where he graduated with honors, and an M.Sc. in Operations Research from the National Autonomous University of Mexico (UNAM). His research interests include mathematical modeling of complex systems, fractional calculus, engineering applications, artificial intelligence, and applied software development. He is a member of Mexico's National System of Researchers (SNII) and has participated in applied research projects involving agriculture, robotics, automotive systems, aerospace technology, and engineering innovation.

LinkedIn Eduardo De La Vega

ACKNOWLEDGMENTS

To my grandmother, Blanca, for being the inspiration for this project and always inspiring me. To Katya for accompanying me every step of the way in all my projects.

REFERENCES

[1] Z. Xue, Y. Song, Z. Mi, X. Zheng, Y. Xia, and H. Chen, "PowerInfer-2: Fast large language model inference on a smartphone," arXiv:2406.06282, 2024.

[2] D. Macario, H. Seferoglu, and E. Koyuncu, "Model-distributed inference for large language models at the edge," arXiv:2505.18164, 2025.

[3] Google, "Pixel 9a technical specifications," Google Store. [Online]. Available: https://store.google.com/product/pixel_9a_specs. Accessed: Oct. 23, 2026.

[4] G. Gerganov et al., "llama.cpp: LLM inference in C/C++," GitHub repository. [Online]. Available: https://github.com/ggml-org/llama.cpp. Accessed: Oct. 23, 2026.

[5] A. Yang et al., "Qwen2.5 Technical Report," arXiv:2412.15115, 2024.

[6] NVIDIA Corporation, "GeForce RTX 5090," NVIDIA. [Online]. Available: https://www.nvidia.com/en-us/geforce/graphics-cards/50-series/rtx-5090/. Accessed: Oct. 23, 2026.

[7] NVIDIA Corporation, "NVIDIA RTX 6000 Ada Generation," NVIDIA. [Online]. Available: https://www.nvidia.com/en-us/products/workstations/rtx-6000/. Accessed: Oct. 23, 2026.

[8] NVIDIA Corporation, "NVIDIA RTX PRO 6000 Blackwell Workstation Edition," NVIDIA. [Online]. Available: https://www.nvidia.com/en-us/products/workstations/professional-desktop-gpus/rtx-pro-6000/. Accessed: Oct. 23, 2026.

[9] NVIDIA Corporation, "NVIDIA RTX PRO 6000 Blackwell Workstation Edition," NVIDIA Marketplace. [Online]. Available: https://marketplace.nvidia.com/en-us/enterprise/laptops-workstations/nvidia-rtx-pro-6000-blackwell-workstation-edition/. Accessed: Oct. 23, 2026.

[10] NVIDIA Corporation, "NVIDIA H100 Tensor Core GPU," NVIDIA. [Online]. Available: https://www.nvidia.com/en-us/data-center/h100/. Accessed: Oct. 23, 2026.

[11] NVIDIA Corporation, "NVIDIA H200 Tensor Core GPU," NVIDIA. [Online]. Available: https://www.nvidia.com/en-us/data-center/h200/. Accessed: Oct. 23, 2026.

[12] S. G. Wani, D. Ellison, and J. Upton, "On-Premise vs Cloud: Generative AI Total Cost of Ownership (2026 Edition)," Lenovo Press, LP2368, Jul. 2026. [Online]. Available: https://lenovopress.lenovo.com/lp2368-on-premise-vs-cloud-generative-ai-total-cost-of-ownership-2026-edition. Accessed: Oct. 23, 2026.

[13] K. Zhang, H. He, S. Song, J. Zhang, and K. B. Letaief, "Distributed on-device LLM inference with over-the-air computation," arXiv:2502.12559, 2025.